SIGma **Fall 2026, October 29th, 2026**

2027 CG:SHOP: lawn mowing with circles

Lecturer: Ian Chen *Scribe: Ian Chen*

Now, we want to generalize lawn-mowing to shapes that are not necessarily a unit-square cutter. Remember that this unit-square cutter allowed us to reduce our problem to TSP, so that the order of visiting the grid exactly specifies the tour. For other shapes (namely, a circle), it is not as clear, and moreover, we would have no idea how to relate such a path to the optimal solution.

As such, we have two problems to tackle:

- how do we turn tours of a unit-square into tours of other shapes (the unit-circle)
- and how do we bound the approximation ratio of such a procedure?

I want to briefly emphasize, once again, that this is hard![1] The optimal circle-cutter tour can do anything: not axis-aligned, and even the optimal tour has endpoints that are incredibly difficult to calculate (as discussed in [2], roots of 16th-degree polynomials[2]). Compare to the case last week, where everything was integers, and it worked out much nicer.

key observation: Our avenue forward will be to consider local "patterns" of the unit-square tour, and figure out how to cover those patterns with the unit-circle. Then, we can lower-bound the optimal unit-circle tour by the optimal unit-square tour, which we already have tools to analyze (from last week, namely via the traveling salesman problem).

1 TSP on grid graphs

The particular graph formed is called a grid-graph, where each vertex is a point on a grid and is connected to adjacent points. Being highly regular, we can actually do better than the 2-approx for TSP that we discussed 2 weeks ago. This is the main gadget that drives us towards good approximations; these were first presented in [1].

1.1 no holes

Suppose G has no cut vertices (i.e. bridges where removing a vertex separates it) or holes. We don't address cut-vertices, as we can easily find optimal tours of each part individually after removing the bridge. Removing holes is a simplifying assumption we will return back to later.

Now, we will prove that there exists a tour of length $\leq (6N-4)/5$. Moreover, we can find this in time $O(n)$, where n is the number of sides of the rectilinear polygon. Note that consequently, this is related to the approximation factor:

Corollary: This is a $6/5$-approx to the TSP.

Proof: Any tour visits all N nodes, so N is a lower bound of the optimal.

$$\text{approx-ratio} = \max_R \frac{|\mathcal{A}(R)|}{\min_{A'}|A'(R)|} \leq \max_R \frac{(6|R|-4)/5}{|R|} \leq \frac{6}{5}$$

Now, we prove the $(6N-4)/5$ bound on the tour.

[1] that doesn't mean the state-of-the-art is difficult to understand, but just appreciate the ingenuity that comes into coming up with such ideas.

[2] the algebraic complexity of circles is also cited as why CG:SHOP formulated instances with integers only.

1. remove cut vertices (they split the tour into disjoint parts).

as we said, this is a pre-processing step, and the bound applies only to graphs without cut vertices. However, splitting the tour into disjoint parts via cut vertices is indeed optimal, so we lose nothing in our approximation factor.

2. remove bottlenecks: two opposite sides of a unit square, as in Fig. 1.

Indeed, removing bottlenecks does not cost anything asymptotically: these are the first of what we call "free" transformations. Splitting the graph apart leads to $N_1 + N_2 = N + 4$ (overhead of 4 nodes), but we also remove a net of 4 edges. This fundamental charging argument is going to be central to this analysis.

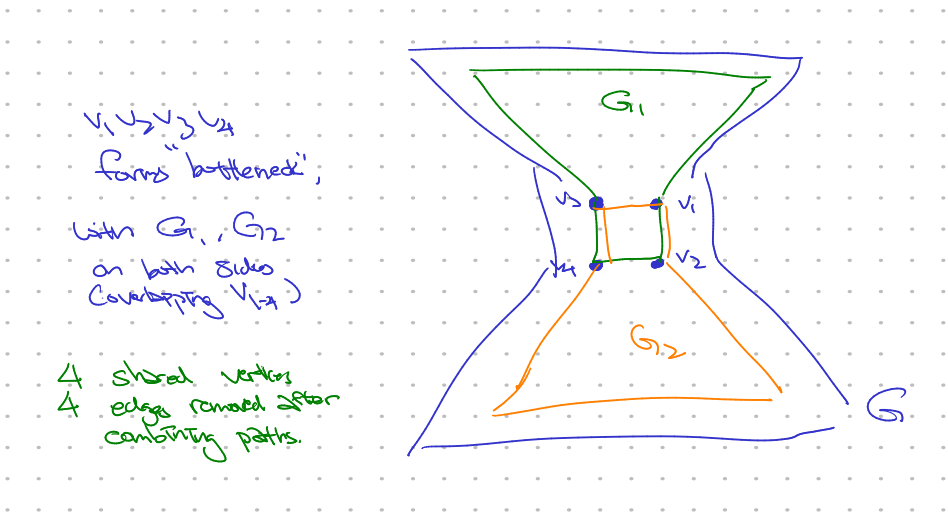


Figure 1: **Removing a bottleneck:** If $v_{1,2,3,4}$ is a bottleneck, form G_1 and G_2 where $G_1 \cap G_2 = \{v_1, v_2, v_3, v_4\}$ and cover G. A tour can be found in both independently, and combined by dropping the 4 duplicate edges (square of v_{1-4}).

3. gradually extend C locally to fill in the interior of G. There are two free detours, Fig. 2.

And then, there are 6 nonfree cases, which are exhaustively checked (we leave these to the paper). We argue that each one wastes k edges, and can be charged to $5k$ nodes, showing an approx-ratio of $6/5$.

1. If one isolated node is missed, we waste 1 edge, and this can occur once every 5 nodes (Fig. 3)
2. We waste 2 per 10 nodes (Fig. 3). Note that if the two internal nodes are opposite an edge of the contour, then we do a type (b) detour.
3. Remaining 4 cases described; c.f. Figure 15 of [1].

More precisely, if there are n nodes, then there can be at most $n/5$ wasted edges, because each wasted edge is mapped in a $1:5$ ratio injectively onto n. Therefore, it costs $6n/5$ and we get the desired approximation ratio.

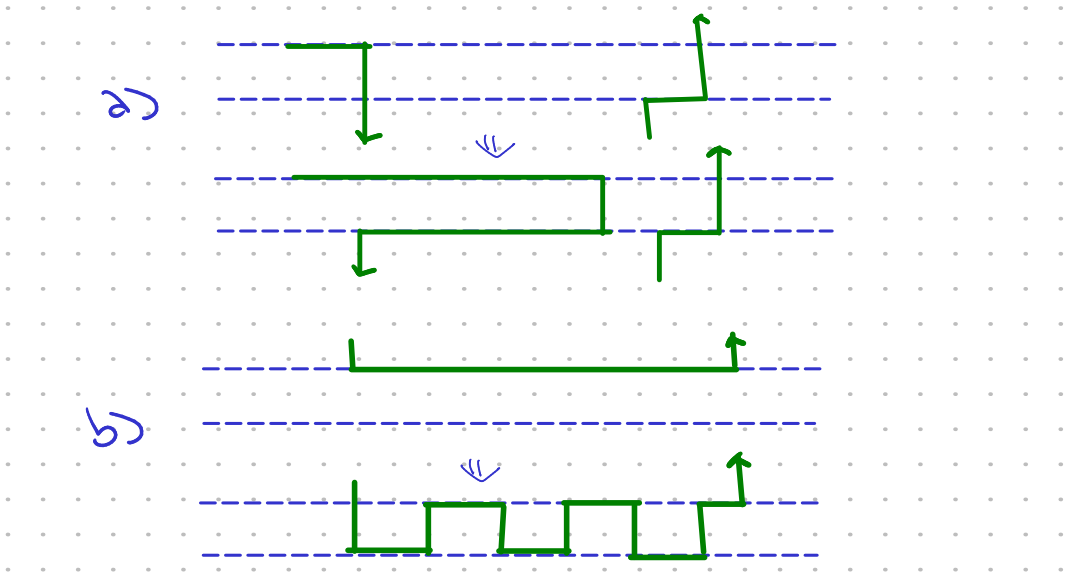


Figure 2: **Free transformations:** Filling in the inside of a polygon with free transformations.

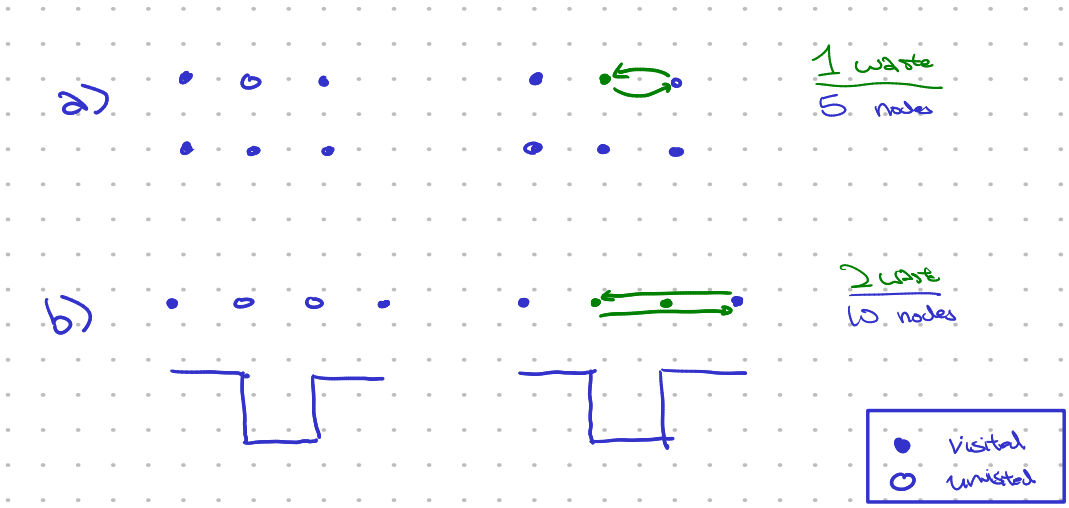


Figure 3: **Free transformations:** Filling in the inside of a polygon with free transformations.

1.2 holes

If the graph has holes, a similar recipe can be used (which we briefly sketch here).

1. do free merges (as in Fig. 2)
2. subtours have size at least 16, so we get $(6/5)N + 2(N/16) = 53N/40$

There are 8 per subtour, and merging charges 16. A subtour must contain 2 in each side, as in Fig. 4.

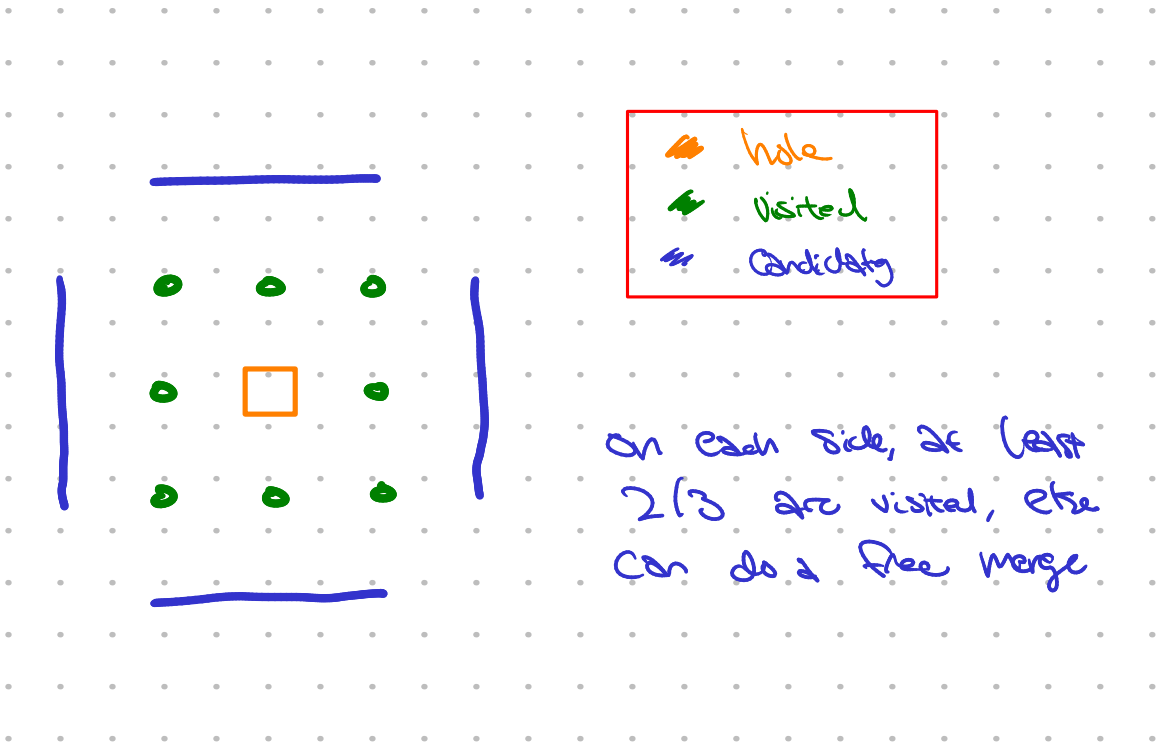


Figure 4: **Subtours have size at least** 16**:**

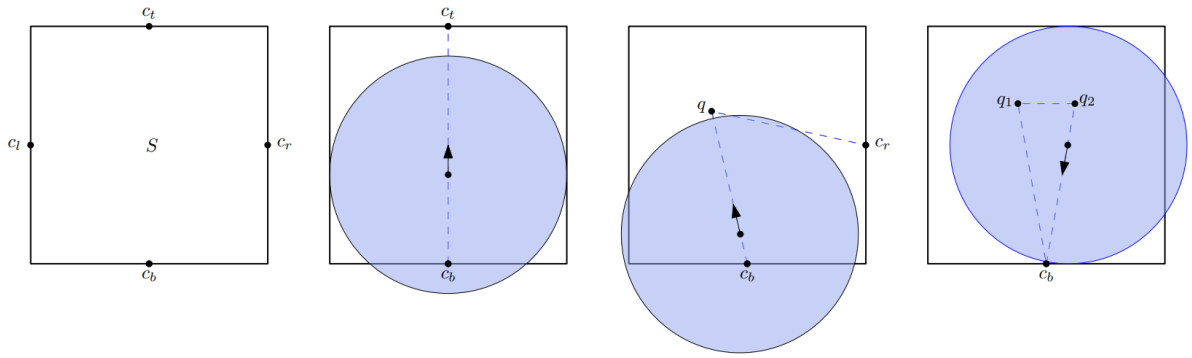
2 circles

Now, for the circle, the proposed algorithm is to take the grid and perturb, as in Fig. 5 (also assuming that transition points are at the center):

i) straight, with cost 1

ii) turn, with cost $1 + \tau$

iii) U-turn, with cost $1 + \sigma$



(a) A pixel S with transition points. **(b)** A straight-line covering, with zero turn **(c)** A covering for a single, 90-degree turn **(d)** A covering for a double, 180-degree U-turn

Figure 5: From Fig. 4 of [2].

Here, $\tau \approx 0.32566$ and $\sigma \approx 0.61118$ [2]. Moreover, for U-turns, the 2nd visit to the pixel after does not need to get perturbed, so the cost is a $(1 + \max(\tau, \sigma/2))$-factor. As $\sigma < 2\tau$, this is a $(1 + \tau)$-factor.

3 conclusion

Here, we have shown that we can achieve a $(6/5)(1 + \tau)$-factor approximation for polygons without holes, or a $(53/40)(1 + \tau)$-factor approximation with holes, for unit circles cutters on axis-aligned integer-coordinate polygons. Namely, this cutter can have arbitrary motion, and we exploit this to create efficient local patterns to cover the grid traversal. Our analysis relied on a "charging" argument: we bound the number of "wasteful" edges by mapping them injectively to vertices, and with a $5 : 1$ ratio, we get an approx factor of $6/5$. This general strategy is broadly applicable, not only in approximation, but more commonly in runtime (especially amortized) analysis.

I think that this is quite a simple but powerful technique about finding patterns; the paper uses each turn or straight-line through a tile, but it seems like bigger tiles could be better (at the cost of more computation and analysis). Moreover, there seem to be many ways that we can apply these techniques; some immediate thoughts of mine are:

- Why can't we extend this to other shapes?
- We could first find a tour with an easier shape, and then break it down using patterns.
- Do we compute these constants for polyominoes?

References

[1] Esther M. Arkin, Sándor P. Fekete, and Joseph S.B. Mitchell. "Approximation algorithms for lawn mowing and milling". In: *Computational Geometry* 17.1 (2000), pp. 25–50. ISSN: 0925-7721. DOI: https://doi.org/10.1016/S0925-7721(00)00015-8.

[2] Sándor P. Fekete et al. *The Lawn Mowing Problem: From Algebra to Algorithms*. 2023. arXiv: 2307.01092 [cs.CG].